



RU

2021

1 (a)

▼ Explain weighted, non-weighted and unit-distance codes for representing decimal number with examples

1. **Weighted Code:** Weighted code is a code in which each and every digit is having a fixed weight according to its position. Example 8421, 5211, 2421 code

Decimal	8421	5211	2421
0	0000	0000	0000
1	0001	0001	0001
2	0010	0011	0010
3	0011	0110	0011
4	0100	0111	0100
5	0101	1000	0101

2. **Non-Weighted Code:** In non-weighted code, each and every digit is not having a fixed weight. Example gray code, excess-3 code etc.

For example, decimal number 453 can be represented in excess-3 code as, 0111 1000 0110. *first, find their bcd. second, add 3(0011) to each bcd*

3. **Unit Distance Code:** Unit-distance codes, also known as gray codes, are a type of non-weighted code where adjacent numbers differ by only one bit. These codes are often used in digital systems to avoid errors caused by multiple signal transitions when converting between binary codes.

For example, the decimal numbers 0 through 5 can be represented in a unit-distance code such as the binary reflected gray code:

Decimal	Binary	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111

1 (b)

▼ Given two 8-bits 2's complement binary number $A = (00101101)_2$ and $B = (11011010)_2$. Compute $C = A - B$ and verify the result by converting A, B and C into decimal #revised-1

To compute $C = A - B$ for the given 8-bit 2's complement binary numbers $A = (00101101)_2$ and $B = (11011010)_2$, follow these steps:

Step 1: Find the 2's complement of B.

- Invert all the bits of B.

$$B = (11011010)_2$$

1's complement of B =

$$(00100101)_2$$

- Add 1 to the LSB (Least Significant Bit) of the 1's complement.

$$C = (00100101)_2 + 1$$

$$C = (00100110)_2$$

Step 2: Perform binary addition of A and the 2's complement of B (C).

$$A = (00101101)_2$$

$$C = (00100110)_2$$

$$C = (+38)_{10}$$

Verification:

To verify the result, convert A, B, and C into decimal.

$$A = (00101101)_2 = (+45)_{10}$$

$$B = (11011010)_2 = (-38)_{10}$$

$$C = (00100110)_2 = (+38)_{10}$$

Now, compute $A - B$ in decimal:

$$A - B = (+45)_{10} - (-38)_{10} = (+45)_{10} + (+38)_{10} = (+83)_{10}$$

Since C in binary and $A - B$ in decimal both yield the same value of $(+83)_{10}$, the computation is verified to be correct.

Therefore, $C = A - B = (00100110)_2$, and it corresponds to the decimal value $(+38)_{10}$

1 (c)

- ▼ How parity bit can be used for single-bit error detection? Explain with an example

The concept of a parity bit is based on the idea of adding an extra bit to a binary data word to ensure that the total number of 1s in the word (including the parity bit) is either odd or even

Let's consider a 4-bit data word, goal is to add a parity bit to the 4-bit data word such that the total count of 1s becomes even

Data word: 1101

Number of 1s in the data word = 3 (odd count)

Since there are 3 1s in the data word, we need to add a parity bit value of 1 to make the total count of 1s even

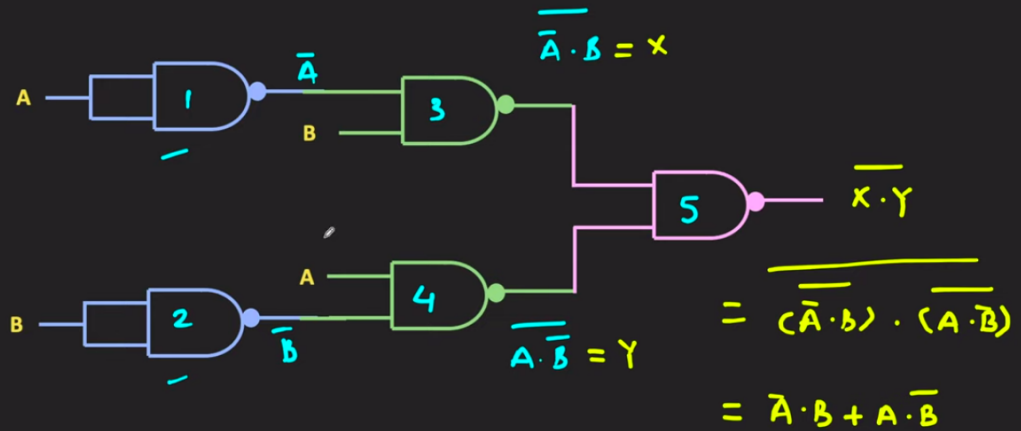
Data word with even parity: 11011

2 (a)

- ▼ Draw a circuit diagram to implement an Exclusive-OR gate using NAND gates

NAND Gate as Universal Gate

XOR Gate using NAND Gates



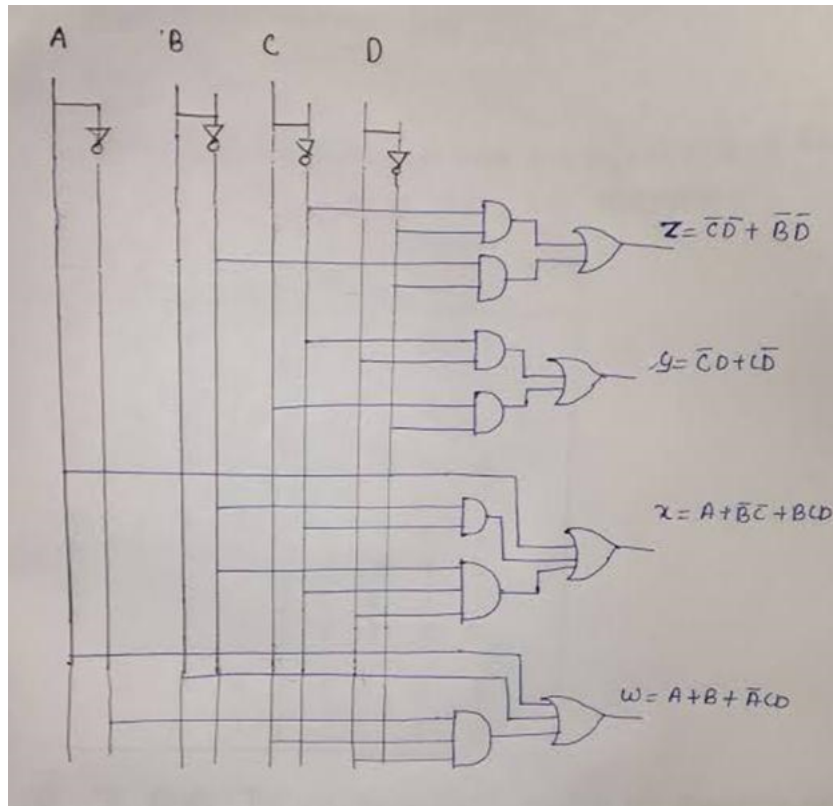
ALL ABOUT ELECTRONICS



2 (b)

▼ Design a circuit diagram for converting BCD code into 2-out-of-5 code

Decimal Digit	BCD 8-4-2-1	6-3-1-1	Excess-3	2-out-of-5 6-3-2-1-0	Gray
0	0000	0000	0011	00011	0000
1	0001	0001	0100	00101	0001
2	0010	0011	0101	00110	0011
3	0011	0100	0110	01001	0010
4	0100	0101	0111	01010	0110
5	0101	0111	1000	01100	0111
6	0110	1000	1001	10001	0101
7	0111	1001	1010	10010	0100
8	1000	1011	1011	10100	1100
9	1001	1100	1100	11000	1101



2 (c)

Prove

▼ i) $y + x\overline{y}z + xz = x + y$

Steps

Start

$$Y + X\overline{Y}Z + XZ$$

Apply the Distributive Law: $AB + AC = A(B + C)$

$$Y + X(\overline{Y}Z + Z)$$

Apply the Absorption Law: $\overline{A}B + A = B + A$

$$Y + X(\overline{Y} + Z)$$

Apply: Distribution

$$Y + X\overline{Y} + XZ$$

Apply the Absorption Law: $\overline{A}B + A = B + A$

$$Y + X + XZ$$

Apply the Absorption Law: $A + AB = A$

$$Y + X$$

▼ ii) $\overline{(x+y)}z + x\bar{y} = \bar{y}(x+z)$

Left Hand Side:

Steps

Start
 $\overline{X+Y}Z + X\bar{Y}$

Apply: Demorgan Theorem
 $\bar{X}\bar{Y}Z + X\bar{Y}$

Apply the Distributive Law: $AB+AC = A(B+C)$
 $\bar{Y}(\bar{X}Z + X)$

Apply the Absorption Law: $\bar{A}B + A = B + A$
 $\bar{Y}(Z + X)$

Apply: Distribution
 $\bar{Y}Z + \bar{Y}X$

Right Hand Side:

Steps

Start
 $\bar{Y}(X+Z)$

Apply: Distribution
 $\bar{Y}X + \bar{Y}Z$

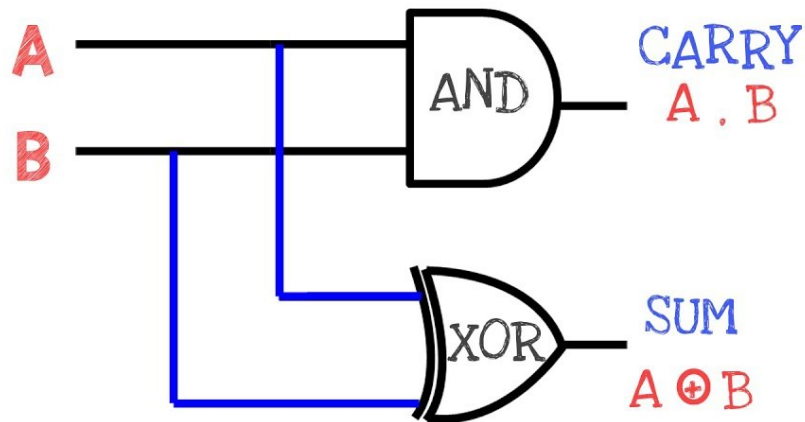
3 (a)

▼ i) Write down the truth table for a half adder

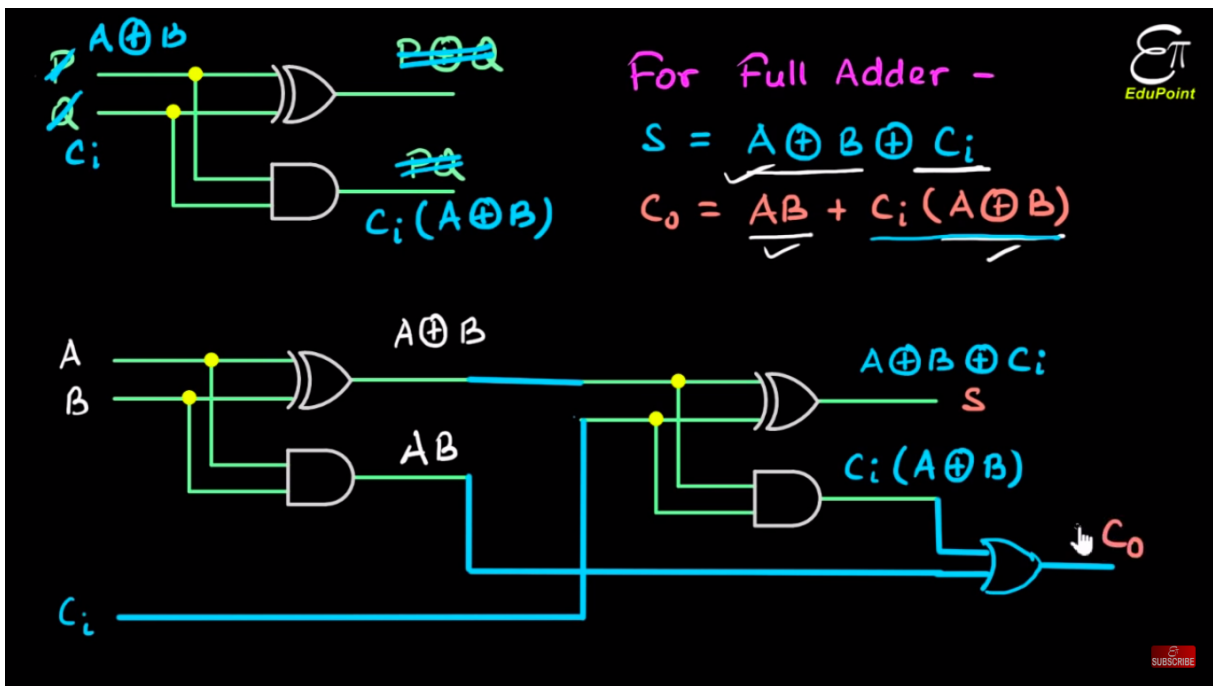
Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

▼ ii) Derive the logic circuit from the truth table of a half adder

Half Adder



▼ iii) Draw a full adder circuit by using two half adder



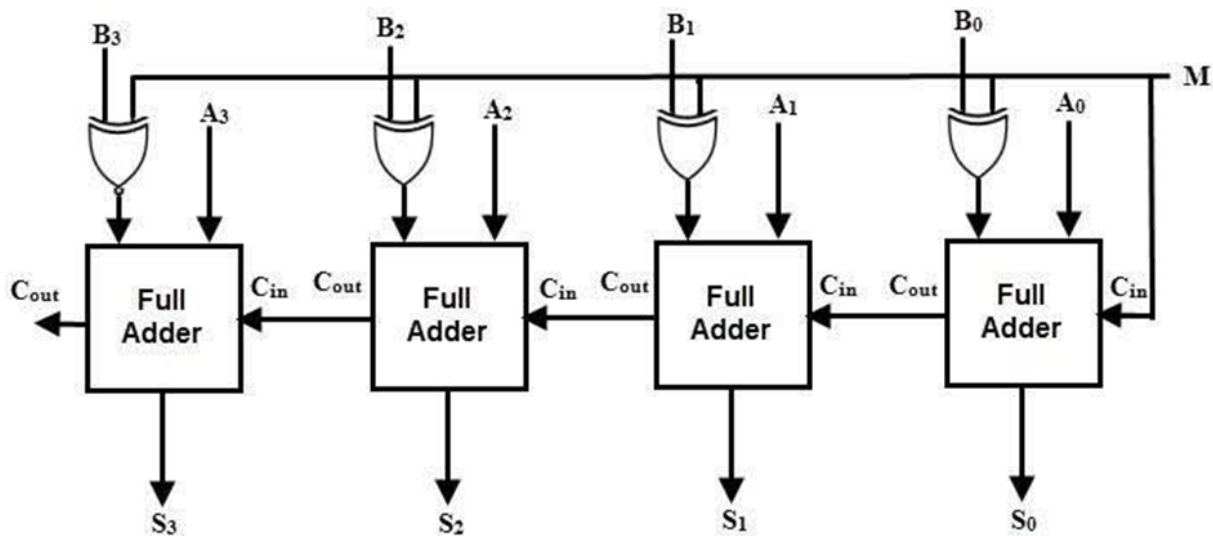
3 (b)

▼ What is the basic difference between a parallel adder and a serial adder? Draw the timing diagram of a 4-bit parallel adder

Aspect	Parallel Adder	Serial Adder
Addition Operation	Performs addition of multiple bits simultaneously.	Performs addition of bits one at a time.
Bit Processing	All bits of the numbers are processed in parallel.	Bits are processed sequentially.

Hardware Requirement	Requires multiple full adders for parallel processing.	Requires fewer hardware resources.
Speed	Faster than serial adders as all additions are done in parallel.	Slower compared to parallel adders.
Suitability	Suitable for adding large binary numbers.	Suitable for applications with limited resources.
Examples	Ripple Carry Adder, Carry Look-Ahead Adder.	Serial-In Serial-Out (SISO) Adder.

Serial Adders process one bit at a time, while Parallel Adders process all bits simultaneously. The choice of which type of adder to use depends on the specific application, and the designer must balance factors such as speed, complexity, and power consumption



3 (c)

- ▼ Implement a logic circuit for a 1-bit comparator and draw its truth table and circuit diagram of a 1-bit binary comparator

<https://images.app.goo.gl/pZfPg7ZxAPC9gMks7>

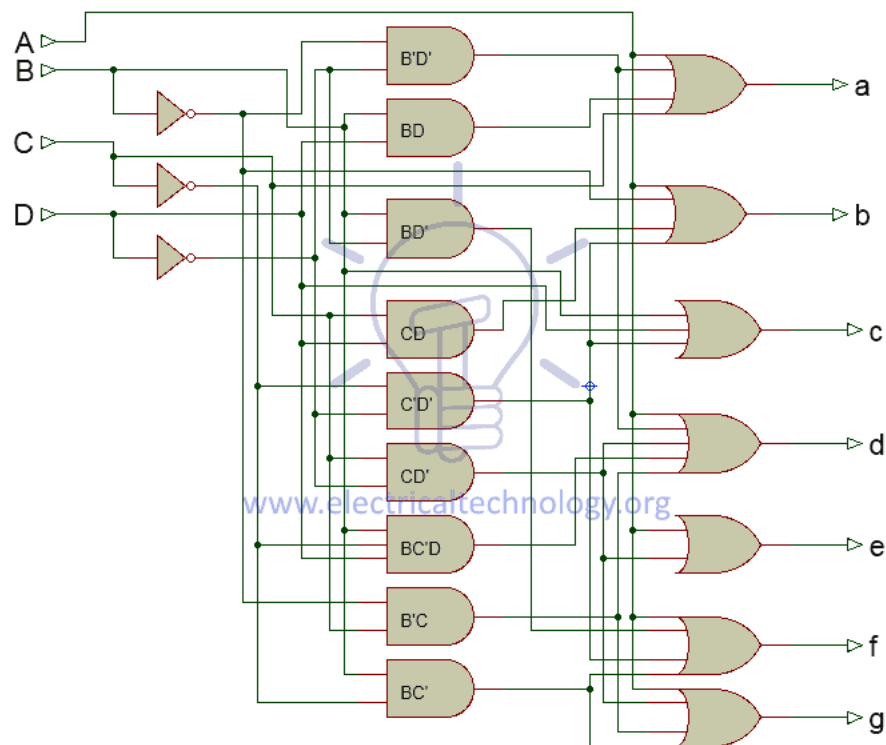
<https://images.app.goo.gl/LvfVRUaSKHmCfCMM8>

4 (a)

- ▼ Design the circuit diagram of BCD to 7-segment decoder

Truth Table

X3	X2	X1	X0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1
1	0	1	0	x	x	x	x	x	x	x
1	0	1	1	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x



Schematic of BCD to 7-Segment Decoder

https://www.youtube.com/watch?v=smeUN1Bxj3M&ab_channel=NesoAcademy

https://www.youtube.com/watch?v=qGFpgpqf7s&t=14s&ab_channel=NesoAcademy

4 (b)

- ▼ Implement the following Boolean expressions using a 74LS138 3-to-8-line decoder $Y_1 = \bar{A}\bar{B} + AC + \bar{A}\bar{C}$, $Y_2 = \bar{A}C + A\bar{C}$, $Y_3 = \bar{B}C + B\bar{C}$

4 (c)

- ▼ Write some typical applications of multiplexer and demultiplexer

Multiplexer	Demultiplexer
Data transmission and communication systems	Data demultiplexing in communication systems
Address decoding in memory systems	Address decoding in memory systems
Audio and video routing in multimedia systems	Signal routing and distribution in audio and video systems
Logic function implementation and signal processing	Implementing logic functions and signal processing

5 (a)

- ▼ Differentiate between i) combinational and sequential logic circuits, ii) synchronous and asynchronous logic circuits

Feature	Combinational Logic Circuit	Sequential Logic Circuit
Output	Depends only on current input values	Depends on current input values and current state
Feedback	No feedback loop	Contains feedback loop
Memory	No memory elements	Contains memory elements (e.g. flip-flops)
Timing	Output is produced instantly after input changes	Output may be delayed due to circuit propagation or clock cycle
Design	Simple design	More complex design
Implementation	Uses logic gates and other combinational components	Uses both combinational and sequential components (e.g. flip-flops)
Applications	Arithmetic circuits, Boolean logic circuits, multiplexers, etc.	Counters, shift registers, finite state machines, etc.

Feature	Synchronous Logic Circuit	Asynchronous Logic Circuit
Clock Signal	A clock signal is used to synchronize all flip-flops and gates	No clock signal is used
Timing	Output depends on clock cycle	Output may occur at any time and can be unpredictable
Design Complexity	More complex design	Can be simpler than synchronous circuits
Delay	Delay is predictable and consistent	Delay is often unpredictable and inconsistent
Power Consumption	Higher power consumption due to clock signal	Lower power consumption due to lack of clock signal
Applications	Processors, digital systems with precise timing requirements	Simple control circuits, low-power devices

5 (b)

- ▼ Define setup time and hold time with necessary diagram

The setup time is the minimum amount of time that the input signal must remain stable before the clock edge in order for the flip-flop to capture the intended value.

Hold time is the minimum amount of time that the input signal must remain stable after the clock edge in order for the flip-flop to reliably hold the captured value.

5 (c)

- ▼ Explain D flip-flop with asynchronous Preset and Clear input signals with block diagram, truth table, and timing diagram

6 (a)

- ▼ Mention the different modes of operations of the shift register. Draw the block diagram of 4-bit serial input serial output shift register, discuss its operation, and draw the truth table and waveforms

Different modes of operations of the shift register:

1. Serial-In, Serial-Out (SISO):

In SISO mode, data is serially input bit by bit from one end and shifted out serially bit by bit from other end.

2. Serial-In, Parallel-Out (SIPO):

In SIPO mode, data is entered into the shift register serially, but it is output in parallel form.

3. Parallel-In, Serial-Out (PISO):

In PISO mode, data is loaded in parallel into the shift register through multiple inputs (parallel inputs). The data is then shifted out serially, one bit at a time, with each clock pulse.

4. Parallel-In, Parallel-Out (PIPO):

In PIPO mode, data is loaded in parallel into the shift register through multiple inputs (parallel inputs). The data is retained in the register until new data is loaded or shifted out. The stored data is available on the parallel output lines.

5. Bidirectional Shift Register:

A bidirectional shift register allows shifting of data in both directions: left and right. It can be controlled by a control signal that determines the direction of the shift.

6. Universal Shift Register:

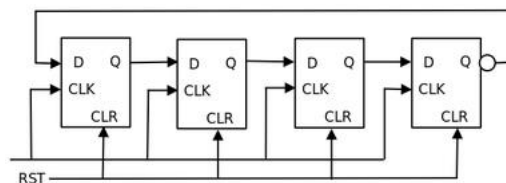
A universal shift register combines the functionalities of different shift register modes. It allows shifting of data in both directions and supports various modes of operation, including SISO, SIPO, PISO, and PIPO.

6 (b)

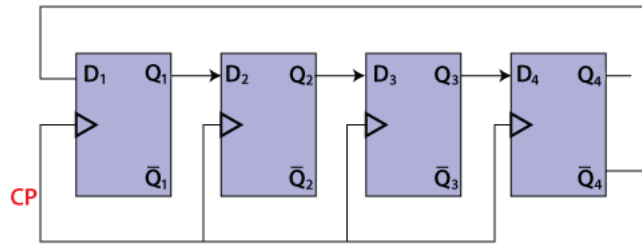
- ▼ Classify counters in shift registers. Explain the operation of Johnson Counter in brief

Basically, counters in shift registers are classified into two types such as ring counter as well as Johnson counter.

Ring Counter: A ring counter is a type of circular shift register that forms a closed loop, where the output of the last flip-flop is connected to the input of the first flip-flop.

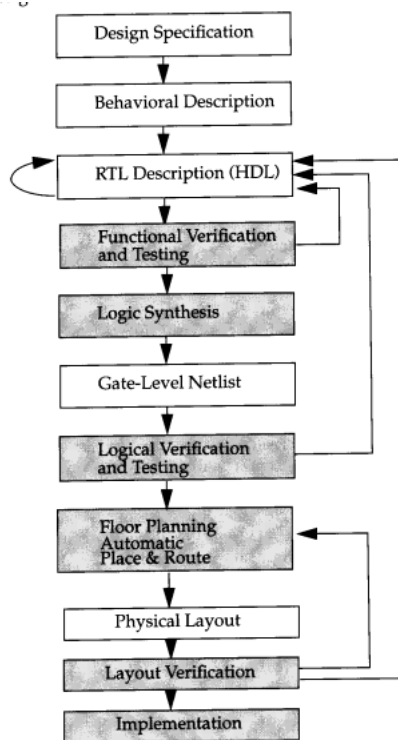


Johnson Counter: The **Johnson counter** is similar to the **Ring counter**. The only difference between the **Johnson counter** and the **ring counter** is that the outcome of the last flip flop is passed to the first flip flop as an input. But in **Johnson counter**, the inverted outcome Q' of the last flip flop is passed as an input.



7 (a)

- ▼ Draw the HDL design flow



7 (b)

- ▼ State the Verilog HDL supported levels of abstraction for designing digital circuits

7 (c)

- ▼ Write HDL code to implement a 4-to-2 line priority encoder

```
module priority_encoder(input [3:0] inputs,
                      output reg [1:0] output);

always @ (inputs) begin
    casez(inputs)
```

```

4'bxxxx: output = 2'b00; //no input
4'b000x: output = 2'b00; //input 0
4'b001x: output = 2'b01; //input 1
4'b01xx: output = 2'b10; //input 2
4'b1xxx: output = 2'b11; //input 3
endcase
end
endmodule

```

8 (a)

- ▼ Explain the port connection rules of Verilog HDL

8 (b)

- ▼ Define rise, fall, and turn-off delays with necessary diagram

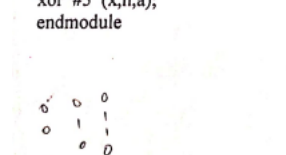
8 (c)

Draw the timing diagram of the following simulate module

```

module comb_circuit(x, v);
input [3:0]v;
output x;
nor #5 (n,v[3],v[2]);
and #10 (a,v[1],v[0]);
xor #5 (x,n,a);
endmodule

```



```

module simulate;
reg [3:0]V; wire X;
comb_circuit(X, V);
initial
begin
V = 4'b0000;
#25 V = 4'b0110;
#25 V = 4'b0011;
#25 V = 4'b1011;
end
endmodule

```

- ▼ Answer

2020

1 (a)

- ▼ Why do we use binary number system instead of decimal number system in the digital devices? **#revised-1**

Binary number system is used in digital devices because it aligns well with the fundamental operation of digital circuits, which use electronic switches to represent data as either 0 or 1.

1 (b)

- ▼ Represent $(93)_{10}$ and $(-93)_{10}$ as 8-bit i) Sign-Magnitude, ii) 1's complement, and iii) 2's complement binary number #2017 (same but different number) **#revised-1**

i) Sign-Magnitude Representation:

To represent

$(93)_{10}$ as an 8-bit sign-magnitude binary number, we first convert the absolute value (93) into binary and then add the sign bit.

$$(93)_{10} = (01011101)_2 \text{ (positive 93 in binary)}$$

Sign-Magnitude representation:

Positive 93: 0 1011101

For the negative value, we use the same binary representation and add the sign bit as 1.

ii) 1's Complement Representation:

To represent

$(93)_{10}$ as an 8-bit 1's complement binary number, we find the 1's complement of the positive binary representation.

1's complement of $(01011101)_2$ is $(10100010)_2$.

1's complement representation:

Positive 93: 0 1011101

Negative 93: 1 0100010

iii) 2's Complement Representation:

To represent

$(93)_{10}$ as an 8-bit 2's complement binary number, we find the 2's complement of the positive binary representation.

2's complement of $(01011101)_2$ is $(10100011)_2$.

2's complement representation:

Positive 93: 0 1011101

Negative 93: 1 0100011

1(c)

▼ State the purpose of Hamming code for digital communication devices. Compute the data word with hamming code for the original binary data word $(10110010110)_2$

Hamming code is a single-bit error detection and correction code. It detects the single bit error in transmitted data and corrects the error

The original data word $(10110010110)_2$

P_1 = XOR of bits (3,5,7,9,11,13,15)

P_2 = XOR of bits (3,6,7,10,11,14,15)

P_4 = XOR of bits (5,6,7,12,13,14,15)

P_8 = XOR of bits (9,10,11,12,13,14,15)

P_1	P_2	3	P_4	5	6	7	P_8
		1		0	1	1	

$$\begin{aligned}
 P_1 &= 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 0 \\
 P_2 &= 1 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1 \\
 P_4 &= 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 0 \\
 P_8 &= 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1
 \end{aligned}$$

Hence the hamming code is,

$$\underline{0} \underline{1} \underline{1} \underline{0} \underline{0} \underline{1} \underline{1} \underline{1} \underline{0} \underline{0} \underline{1} \underline{0} \underline{1} \underline{1} \underline{0} .$$

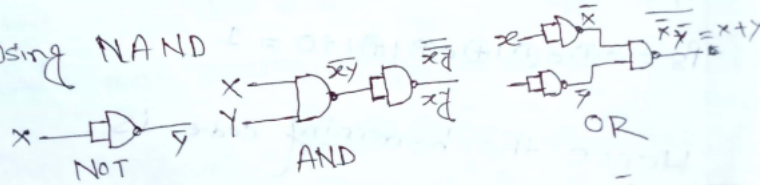
2(a)

▼ Why NAND gate and NOR gate are called universal gate? Explain with necessary diagram

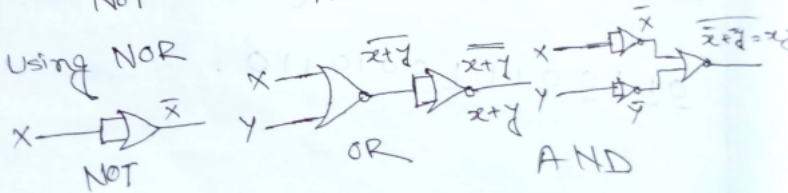
Soln:-

NAND and NOR gates are known as "universal" gate because any digital circuit can be implemented with NAND or NOR gates alone. To prove this, it suffices to show that AND, OR and NOT can be implemented using NAND or NOR gates only.

Using NAND



Using NOR



2 (b)

Prove, using Boolean algebra properties

- ▼ i) $xy + \bar{x}z + yz = xy + \bar{x}z$
- ▼ ii) $\bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} + x\bar{y}\bar{z} = \bar{x}\bar{z} + y\bar{z}$

Soln:-

(i) $x\bar{y} + \bar{x}z + yz$
 $= x\bar{y} + \bar{x}z + yz(x + \bar{x})$
 $= x\bar{y} + \bar{x}z + x\bar{y}z + \bar{x}yz$
 $= x\bar{y}(1 + z) + \bar{x}z(1 + y)$
 $= x\bar{y} + \bar{x}z \text{ (proved)}$

(ii) $\bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} + x\bar{y}\bar{z}$
 $= \bar{x}\bar{y}\bar{z} + \bar{x}y\bar{z} + x\bar{y}\bar{z} + x\bar{y}\bar{z}$
 $= \bar{x}\bar{z}(\bar{y} + y) + y\bar{z}(\bar{x} + x)$
 $= \bar{x}\bar{z} + y\bar{z} \text{ (proved)}$

2 (c)

▼ Simplify the Boolean function $F(A, B, C, D) = \sum(1, 3, 7, 8, 11, 15)$ which has the don't care conditions: $d(A, B, C, D) = (0, 2, 5, 9)$

Soln:-

$F(A, B, C, D) = \sum(1, 3, 7, 8, 11, 15) + d(0, 2, 5, 9)$

AB \ CD	00	01	11	10
00	x	1	1	x
01	0	x	1	0
11	0	0	1	0
10	1	x	1	0

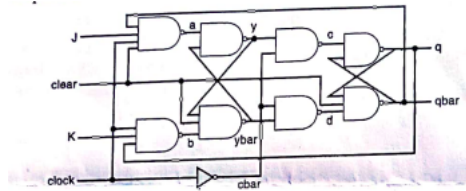
$F(A, B, C, D) = \bar{B}\bar{C} + CD$

3 (a)

▼ Write HDL codes for 2x2-bit combinational array multiplier

3 (b)

- ▼ How to invoke a Verilog module from a VHDL model? Explain with an example of mixed language description or a full adder using 2 half adder



The circuit diagram for the JK flip-flop is given above

4 (a)

- ▼ Implement the above circuit diagram of JK flip-flop as a Verilog HDL module

4 (b)

- ▼ Use the above JK flip-flop module to implement the following 4-bit synchronous counter

5 (a)

- ▼ Implement a full adder circuit using a 3-to-8 decoder

https://www.youtube.com/watch?v=u863cwgdlNlA&ab_channel=NesoAcademy

5 (b)

- ▼ How decoder can be used as de-multiplexer?

A decoder is a digital circuit that converts an input code into one of several possible output codes. A de-multiplexer, or demux for short, is a digital circuit that takes one input and distributes it to one of several possible outputs.

To use a decoder as a de-multiplexer, we can connect the input of the decoder to the input of the demux and connect each output of the decoder to one of the inputs of the demux. The number of outputs on the decoder will determine the number of inputs on the demux.

For example, if we have a 2-to-4 decoder with inputs A and B and outputs Y0 through Y3, we can use it as a 1-to-4 demux by connecting the input of the decoder to our input signal and connecting each output of the decoder to one of the inputs of the demux. We can then select which output of the demux receives the input signal by setting the values of A and B appropriately.

https://www.youtube.com/watch?v=vMvvggyriCc&ab_channel=NesoAcademy

5 (c)

- ▼ How could we implement an 8-to-1 multiplexer using 2-to-1 multiplexer

5 (d)

- ▼ Distinguish between combinational and sequential logic circuit

6 (a)

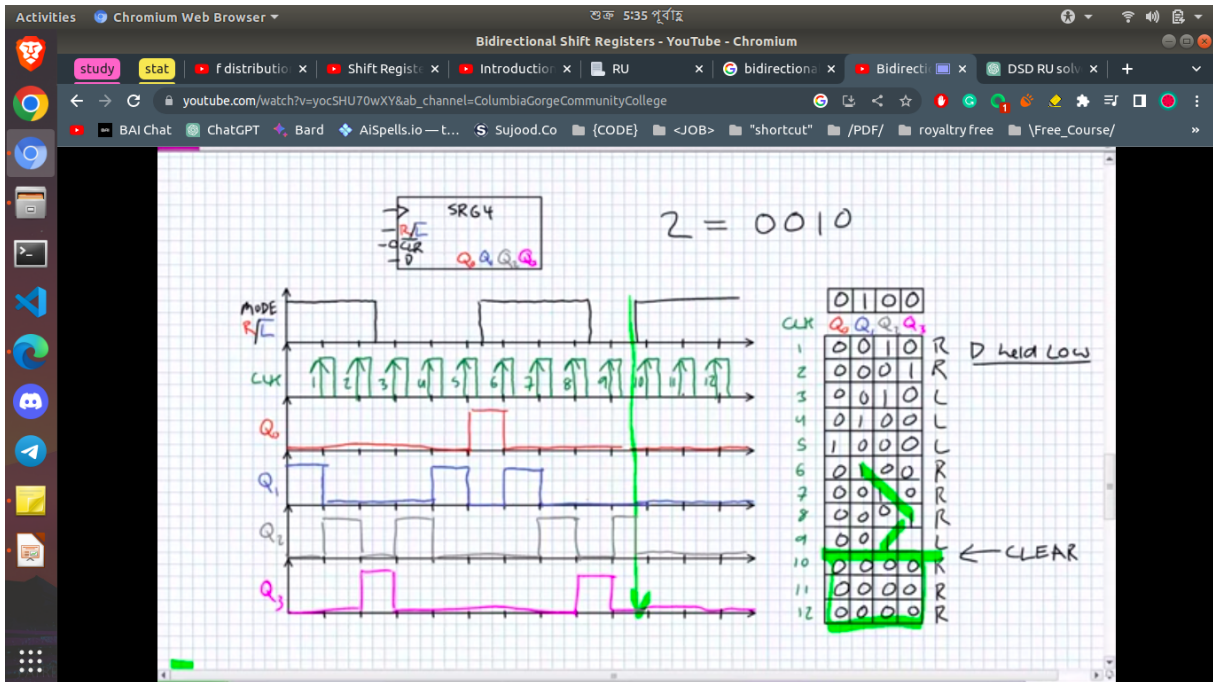
- ▼ Compare between D latch and D flip-flop with timing diagram

6 (b)

- ▼ Show the implementation of T flip-flop from D flip-flop

6 (c)

- ▼ Show the circuit and timing diagram of a 4-bit bidirectional shift register



7 (a)

- ▼ What are advantages of synchronous counter over asynchronous counter?

7 (b)

- ▼ Design a 4 bit synchronous counter for counting the following binary sequence: 0000 → 0101 → 1010 → 0110 → 1001 → 0011 → 1100 → 1111 → 0000

8 (a)

- ▼ How does the binary counter work as a frequency divider?

8 (b)

- ▼ What is the frequency of the output of 8th FF when the input clock frequency is 512 KHz?

8 (c)

- ▼ What is full adder? Design a full adder circuit

2018

1 (a)

▼ Convert 01010010 (BCD) to binary. **#revised-1** You have a BCD number of two digit decimal values. Now design a BCD-to-binary converter with 4-bit parallel adder. Draw the logic circuit and explain its operation

$$(01010010)_{\text{BCD}} = (\underline{\quad? \quad})_2$$

Solution:

$$(01010010)_{\text{BCD}} = (\underline{\quad\quad\quad})_2$$

1. Convert BCD to decimal

$$(01010010)_{\text{BCD}} = (\underline{\quad\quad\quad})_{10}$$

$$\begin{array}{r} 0101 \ 0010 \\ \hline 5 \quad 2 \end{array}$$

$$\therefore (01010010)_{\text{BCD}} = (\underline{52})_{10}$$

2. Convert decimal to binary

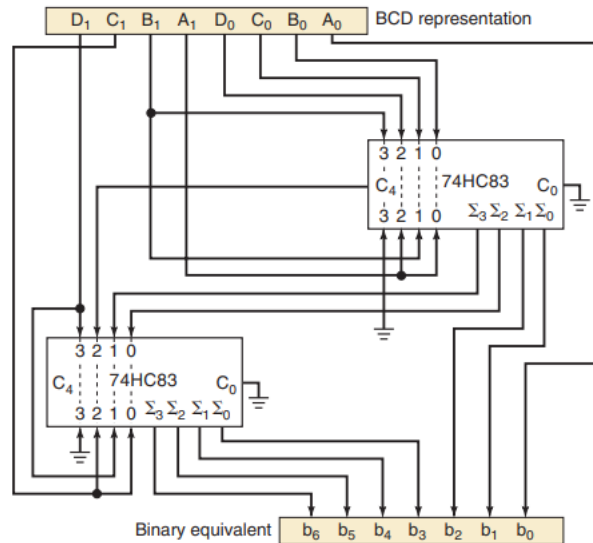
$$(52)_{10} = (\underline{\quad\quad\quad})_2$$

$$\begin{array}{r|l} 2 & 52 \\ \hline 2 & 26 \ 0 \uparrow \\ 2 & 13 \ 0 \uparrow \\ 2 & 6 \ 1 \uparrow \\ 2 & 3 \ 0 \uparrow \\ 2 & 1 \ 1 \uparrow \\ \hline & 0 \ 1 \uparrow \end{array}$$

$$\therefore (52)_{10} = (\underline{110100})_2$$

$$\therefore (01010010)_{\text{BCD}} = (\underline{110100})_2$$

FIGURE 9-43 BCD-to-binary converter implemented with 74HC83 four-bit parallel adders.



The two adder ICs perform the addition of the BCD bits in the proper combinations. The above logic circuit shows that A_0 is the only BCD bit that contributes to the LSB, b_0 , of the binary equivalent. Because there is no carry into this bit position, A_0 is connected directly as output b_0 . The table also shows that only BCD bits B_0 and A_1 contribute to bit b_1 of the binary output. These two bits are combined in the upper-right adder to produce output b_1 . Likewise, only BCD bits D_0 , A_1 , and C_1 contribute to bit b_3 . The upper-right adder combines D_0 and A_1 to generate Σ_2 , which is connected to the lower-left adder, where C_1 is added to it to produce b_3 .

1 (b)

- ▼ Design a BCD to Excess-3 code converter and draw its logic diagram

bcd to excess 3 code converter | code converter | aasaan padhaai

bcd to excess 3 code converter by aasaan padhaai

bcd to xs3 code converter

how to convert bcd code to excess 3

📺 <https://www.youtube.com/watch?v=AoAQJ0dZCH8>

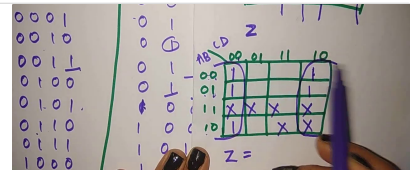


Table 4-1 Truth table for code-conversion example

Input BCD				Output Excess-3 code			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

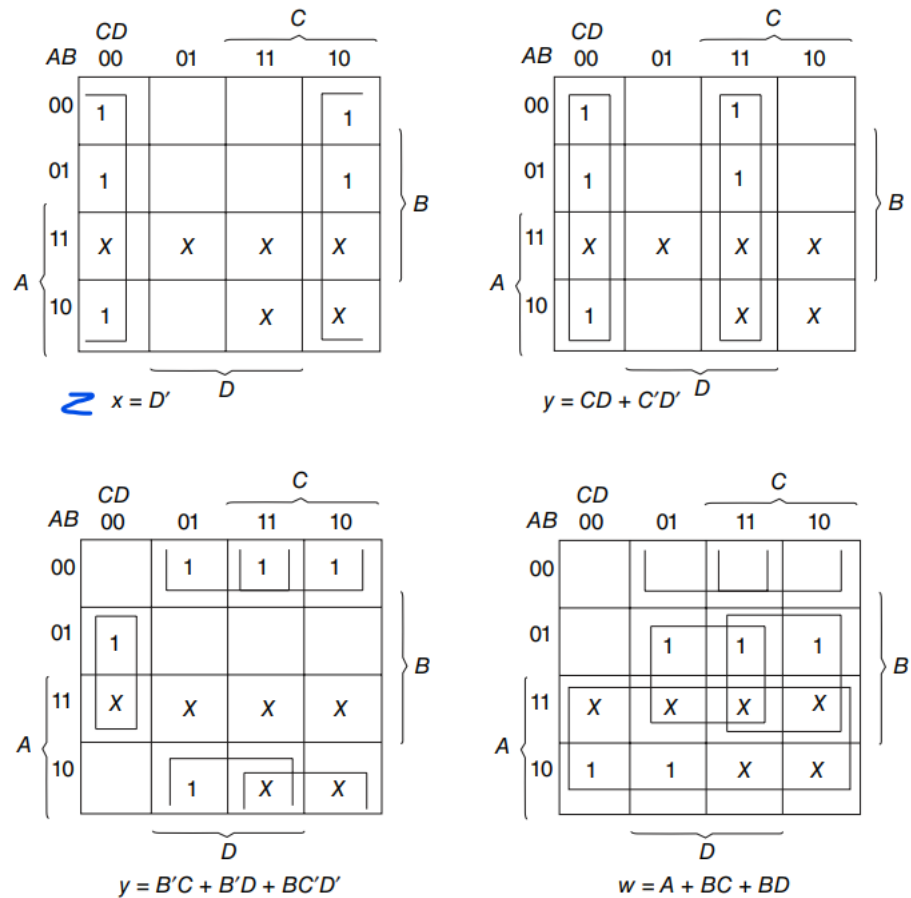


Figure 4.7 Maps for BCD-to-excess-3 code converter

$$\begin{aligned}
 z &= D' \\
 y &= CD + C'D' = CD + (C + D)' \\
 x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\
 &= B'(C + D) + B(C + D)' \\
 w &= A + BC + BD = A + B(C + D)
 \end{aligned}$$

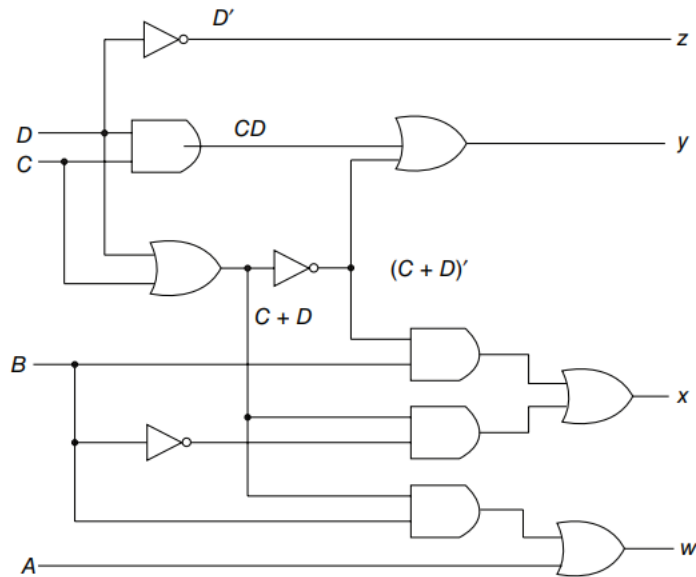


Figure 4.8 Logic diagram for BCD-to-excess-3 code converter

2 (a)

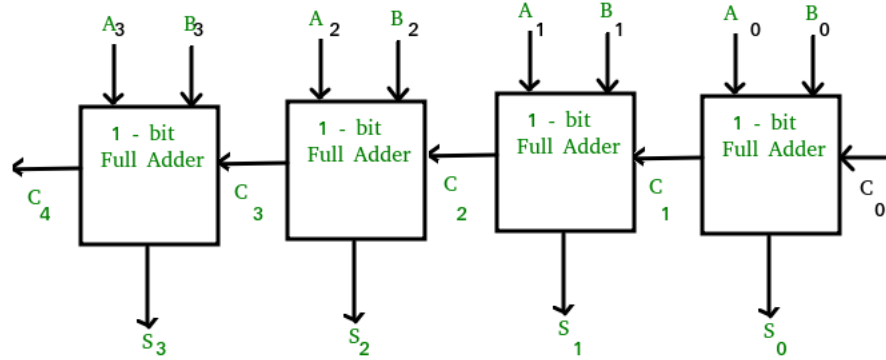
▼ Why carry-look-ahead adder is faster than regular adder-explain with diagram

A carry-look-ahead (CLA) adder is faster than a regular adder because it is designed to perform carry propagation and generation in parallel, whereas regular adders perform these operations sequentially.

When we add two binary digits together, there are three possible outcomes: $0+0=0$, $0+1=1$, and $1+1=10$. In the last case, we get a sum of 0 and a carry of 1. This carry needs to be added to the next column in order to get the correct result.

In a regular adder, each column waits for the carry from the previous column before it can start its own calculation. This means that there is a delay between each step of the addition process, which slows down the overall operation.

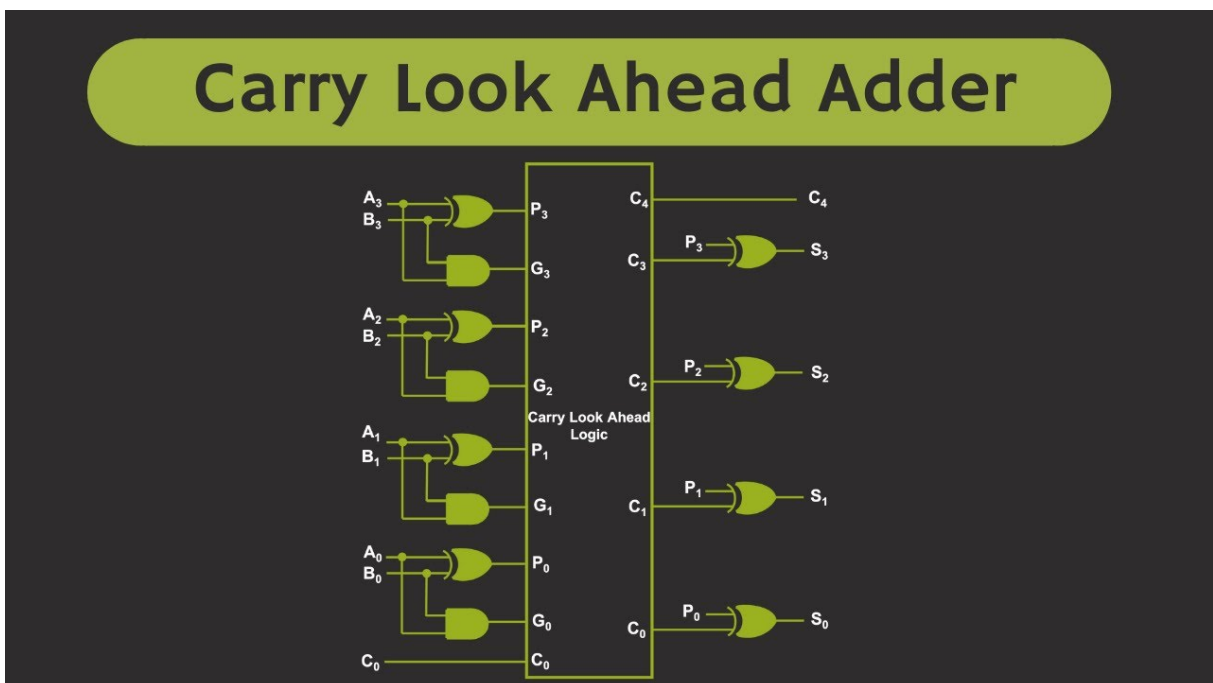
In a carry-look-ahead adder, on the other hand, the carries for all columns are calculated at the same time in parallel. The CLA adder uses a set of logic gates to generate and propagate carries based on the input bits. This allows the adder to compute the sum and carry values for each column without waiting for the previous column to finish. The following diagram shows a 4-bit CLA adder:



As you can see, the carry values are generated and propagated in parallel, which allows the adder to calculate the sum of all four bits at the same time. This results in a much faster operation compared to a regular adder.

In summary, a carry-look-ahead adder is faster than a regular adder because it performs carry propagation and generation in parallel, allowing it to compute the sum of two binary numbers quickly and efficiently.

for knowing purpose:



2 (b)

▼ Distinguish between 1's complement and 2's complement system. Why this circuits are necessary? **#revised-1**

1's Complement	2's Complement
Range: $-(2^{N-1} - 1)$ to $2^{N-1} - 1$	Range: -2^{N-1} to $2^{N-1} - 1$
Zero representation: 0 and -0	Zero representation: Only 0
Number of zeros: Two (positive and negative)	Number of zeros: Only one (positive)
Arithmetic operations: More complex	Arithmetic operations: Simpler and efficient

Digital circuits are necessary because they process digital signals, enable data representation, perform logic operations, handle memory and storage, support computation and control. They are vital for modern electronic devices and systems.

2 (c)

- ▼ Design a logic circuit that has four inputs A,B,C, and D and whose output will be high only when a majority of inputs are high

Let A, B & C be the inputs and Y, the output.

Truth Table

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
✓ 0	1	1	1
1	0	0	0
✓ 1	0	1	1
✓ 1	1	0	1
✓ 1	1	1	1

On simplification

$$Y = AB + BC + AC$$

Circuit

3 (a)

- ▼ Define decoder and demultiplexer. How decoder can be used as a demultiplexer #2020

A decoder is a device that takes in a coded signal and produces an output signal that is a representation of the original input signal. Decoders are used to reverse the process performed by encoders

A demultiplexer (or "DEMUX") is a device that performs the opposite function of a multiplexer. It takes a single input signal and distributes it across multiple output channels based on the states of its select lines.

3 (b)

- ▼ Define Multiplexer. **Implement the logic function $F(A, B, C, D) = \sum m(1, 3, 4, 7, 11, 13, 15)$ using 8×1 MUX**

A multiplexer (or "MUX") is a device that allows multiple input signals to be combined into a single output signal. The selection of which input signal to pass through to the output is controlled by a set of select inputs.

3 (c)

- ▼ Implement the logic function $F(A, B, C) = \sum(1, 3, 4, 7)$ using a 3-to-8 line decoder **with active low output**

https://www.youtube.com/watch?v=rTI7gC8R11A&ab_channel=SuccessGATEway

https://www.youtube.com/watch?v=jME2ITFeQRE&t=382s&ab_channel=amithnayak

4 (a)

- ▼ What is parity bit? Implement a 4-bit even parity generator circuit and discuss the operation of the circuit with example

A parity bit is an extra bit added to a binary data sequence to detect errors during transmission or storage.

<https://www.pcepuirnia.org/wp-content/uploads/2020/03/Parity-Generator-and-Cheker.pdf>

Let the 4-bit incoming bits be A, B, C, D & P_e be the even parity bit.

A	B	C	D	P_e
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1

A	B	C	D	P_e
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

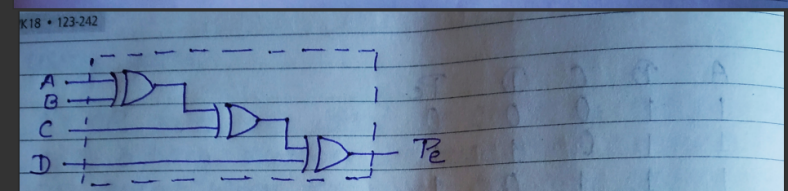
Solving for P_e using K-Map

$\begin{matrix} AB \\ CD \end{matrix}$	00	01	11	10
00		1		1
01	1		1	
11		1		1
10	1		1	

No group formed, So we will have to solve it algebraically

$$\bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}BCD + A\bar{B}\bar{C}D + A\bar{B}C\bar{D} + A\bar{B}\bar{C}D + A\bar{B}CD$$

$$\begin{aligned} & \bar{A}\bar{B}(\bar{C}D + C\bar{D}) + \bar{A}B(\bar{C}\bar{D} + CD) + A\bar{B}(\bar{C}D + C\bar{D}) + A\bar{B}(CD + \bar{C}\bar{D}) \\ & (\bar{A}\bar{B} + \bar{A}B)(\bar{C}D + C\bar{D}) + (A\bar{B} + AB)(CD + \bar{C}\bar{D}) \\ & (\bar{A}\oplus\bar{B})(C\oplus D) + (A\oplus B)(C\oplus D) \\ & A\oplus B\oplus C\oplus D \end{aligned}$$



4 (b)

▼ Define state table and state diagram with example

1. State Table:

A state table, also known as a state transition table, is a tabular representation that describes the behavior of a finite state machine (FSM). It shows the current states, possible inputs, and the corresponding next states and outputs.

Each row in the table represents a unique combination of the current state and input, along with the resulting next state and output.

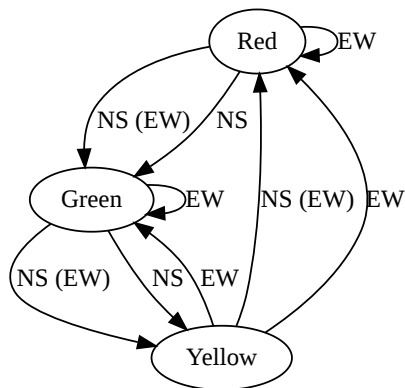
Here is an example of a state table for a simple traffic light controller:

Current State	Input	Next State	Output
Red	NS	Green	GO
Red	EW	Red	STOP
Green	NS	Yellow	CAUTION
Green	EW	Green	GO
Yellow	NS	Red	STOP
Yellow	EW	Red	STOP

In this example, the traffic light controller has three states: Red, Green, and Yellow. The inputs are "NS" (North-South) and "EW" (East-West). The next state column represents the state that the traffic light transitions to based on the current state and input. The output column indicates the corresponding action or output associated with each state.

State Diagram:

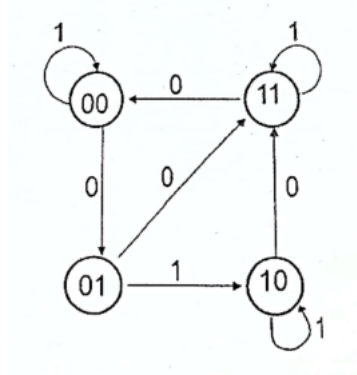
A state diagram is a graphical representation of the states, inputs, and transitions of a finite state machine. It uses circles or rounded rectangles to represent states and arrows to represent transitions between states triggered by inputs. The state diagram provides a visual representation of how the system behaves and transitions from one state to another.



Here, starting from the Red state, if the input is "NS" (North-South), the traffic light transitions to the Green state and the output is "GO". If the input is "EW" (East-West), the traffic light stays in the Red state and the output is "STOP"

4 (c)

From the following state diagram, design the sequential circuit using JK flip-flops



▼ Answer

Excitation Table of JK Flip-Flop

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/eb6a4824-03a2-48df-ac1b-061e6bb34041/from_state_diagram_to_sequential_circuits_using_jk_flip_flop.pdf

5 (a)

▼ Define asynchronous and synchronous counter. Write the advantages of synchronous counter over asynchronous counter

1. Asynchronous Counter:

An asynchronous counter, also known as ripple counter, is a type of digital counter where the flip-flops within the counter are not clocked simultaneously. Each flip-flop receives its clock input from the output of the previous flip-flop in the sequence.

2. Synchronous Counter:

A synchronous counter is a type of digital counter where all the flip-flops within the counter are clocked simultaneously by a common clock signal. This means that all flip-flops change their state simultaneously on the rising or falling edge of the clock signal.

Advantages of Synchronous Counters over Asynchronous Counters:

- Asynchronous counter are serial counters and different flip flops are triggered with different clock pulse while synchronous counter are parallel counters and all the flip flops are triggered with the same clock pulse.
- Only fixed number of count sequence is possible with asynchronous counter while any number of count sequence is possible with synchronous counter.
- There are problems of decoding errors with asynchronous counter while there are no such problems with synchronous counter.

5 (b)

▼ Construct a binary counter that will convert a 64-kHz pulse signal into a 8-kHz square wave

5 (c)

▼ Draw the logic diagram of a Mod-8 synchronous up/down counter. Explain its operation with timing diagram

3 Bit Synchronous Up Counter || MOD 8 Synchronous Up Counter || Synchronous Counter || Counters ||

3 Bit Synchronous Up Counter

3 Bit Synchronous Up Counter using JK Flipflop

MOD 8 Synchronous Up Counter

https://www.youtube.com/watch?v=KxeMWrlIM84&ab_channel=DIVVELASRINIVASARAO

MOD-8
Synchronous
Up Counter

**Design 3
Synchron
Up Cour
MOD.
Synchro
Up Co**

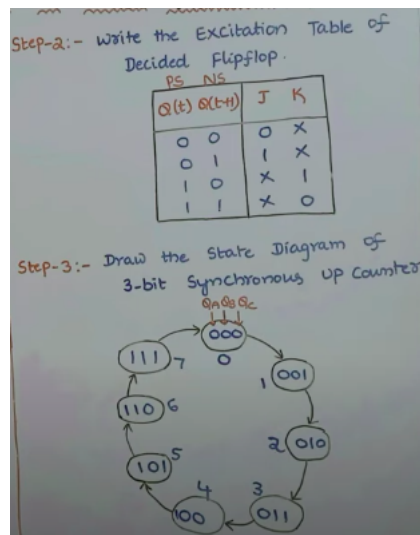
Step-1:- Determine the number of Flipflops needed and Decide which Flipflop we have to use.

$\therefore$ Number of Flipflops required (n) = 3

$\therefore$ Number of States (N) = $2^n = 2^3 = 8$
(0 to 7)

$\therefore$ Maximum Count = $N-1 = 8-1 = 7$

JK/T Flipflops used to Design
= 3-bit Synchronous Up Counter



Step-4:- write the circuit Excitation Table

PS			NS			Excitation Inputs					
Q_A	Q_B	Q_C	Q_A^+	Q_B^+	Q_C^+	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

Step-5:- Find the Simplified Boolean Expression for $J_A, K_A, J_B, K_B, J_C, K_C$ using K-map

for J_A

$Q_B Q_C$	00	01	11	10
Q_A			1	
0				
1	X	X	X	X

$J_A = Q_B Q_C$

(or) MOD-8 Synchronous up counter

for K_A

$Q_B Q_C$	00	01	11	10
Q_A				
0	X	X	X	X
1			1	

$K_A = Q_B Q_C$

for J_B

$Q_B Q_C$	00	01	11	10
Q_A				
0		1	X	X
1		1	X	X

$J_B = Q_C$

for K_B

$Q_B Q_C$	00	01	11	10
Q_A				
0	X	X	1	
1	X	X	1	

$K_B = Q_C$

for J_C

$Q_B Q_C$	00	01	11	10
Q_A				
0	1	X	X	1
1	1	X	X	1

$J_C = 1$

for K_C

$Q_B Q_C$	00	01	11	10
Q_A				
0	X	1	1	X
1	X	1	1	X

$K_C = 1$

DSD
SR Flip Flop

Excitation Table

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

D Flip Flop

Excitation Table

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

JK Flip Flop

Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

T Flip Flop

Excitation Table

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

6 (b)

▼ Design a counter with JK flip flop that will count the sequence 0,1,5,4,3,2,0

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/5c29cafe-1bf5-432b-8d4d-ced79a45e33c/2018_6b.pdf

7 (a)

- ▼ Discuss FPGA architecture in brief. Distinguish between PLA and PAL in terms of design architecture

FPGA Architecture:

FPGA stands for Field-Programmable Gate Array. It is a programmable integrated circuit that allows users to configure and implement digital logic circuits according to their specific requirements. FPGA architecture consists of three main components: configurable logic blocks (CLBs), input/output blocks (IOBs), and interconnect resources.

1. Configurable Logic Blocks (CLBs):

CLBs are the fundamental building blocks of an FPGA. They are composed of lookup tables (LUTs), flip-flops, and multiplexers. CLBs can be interconnected to create complex logic functions.

2. Input/Output Blocks (IOBs):

IOBs provide the interface between the external world and the internal logic of the FPGA. They consist of input and output buffers, programmable I/O standards, and various interfaces such as LVCMOS, LVDS, or GPIO (General Purpose I/O).

3. Interconnect Resources:

Interconnect resources include programmable routing channels and switch matrices. These resources enable the routing of signals between CLBs, IOBs, and other components within the FPGA.

Distinguishing PLA and PAL in Terms of Design Architecture:

Here's a table comparing PLA (Programmable Logic Array) and PAL (Programmable Array Logic) based on their design architectures:

PLA	PAL
Structure: AND plane followed by OR plane	Structure: Programmable AND array followed by OR gate
Programmability: Limited	Programmability: High
Inputs can be combined arbitrarily	Inputs are predefined and fixed
More flexible in terms of inputs	Less flexible due to fixed input structure
Complex circuitry	Simpler design
Can be less efficient	Can be more efficient

7 (b)

- ▼ Implement the Boolean function using PAL $Y_1 = \sum m(1, 2, 4, 7)$ $Y_2 = \sum m(3, 6)$

https://s3-us-west-2.amazonaws.com/secure.notion-static.com/4230e577-7cbe-482a-8635-5d0449e4650e/2018_7b.pdf

7 (c)

- ▼ Determine the size of the PROM required to implement a dual 8-to-1 multiplexer with common selection inputs

The size of the PROM required for implementing an n-input, m-output combinational circuit is $2^n \times m$. For dual 8 to 1 multiplexer, the number of input = (8+8+3 select input)=19 and number of output is 2

Size of PROM = $2^{19} \times 2 = 512K \times 2$

8 (a)

▼ What is VHDL? Briefly describe the salient features of VHDL and Verilog

The VHSIC Hardware Description Language is a hardware description language that can model the behavior and structure of digital systems at multiple levels of abstraction.

Here are some salient features of VHDL:

1. Strong Typing: VHDL has a strong typing system, which means that data types need to be explicitly declared. This helps ensure proper data representation and prevents unintended type conversions or errors.
2. Generics and Parameters: VHDL allows the use of generics and parameters, which are variables that can be defined at the entity level and used throughout the design.
3. Strong Verification Support: VHDL provides built-in constructs for writing testbenches and performing functional verification of digital designs.

Here are some salient features of Verilog:

1. Conciseness: Verilog is known for its concise syntax, which allows designers to describe complex digital systems with fewer lines of code compared to other languages.
2. Event-Driven Modeling: Verilog follows an event-driven simulation model, where changes in signal values trigger events that propagate through the design.
3. Easy Integration with Digital Design Tools: Verilog has good tool support and can be seamlessly integrated into digital design flows. It is widely used in electronic design automation (EDA) tools for synthesis, simulation, and verification.

8 (b)

▼ Explain an 8-to-1 line Multiplexer module in Verilog HDL

Here's an example of an 8-to-1 line multiplexer module in Verilog HDL:

```
module Multiplexer8to1 (
    input [7:0] data,
    input [2:0] select,
    output reg output
);

always @* begin
    case (select)
        3'b000: output = data[0];
        3'b001: output = data[1];
        3'b010: output = data[2];
        3'b011: output = data[3];
        3'b100: output = data[4];
        3'b101: output = data[5];
        3'b110: output = data[6];
        3'b111: output = data[7];
    endcase
end
```

```
end  
  
endmodule
```

8 (c)

- ▼ Write a VHDL code to implement XOR operation

```
library IEEE;  
use IEEE.std_logic_1164.all;  
entity xor_gate is  
    port(A: in std_logic;  
         B: in std_logic;  
         Y: out std_logic);  
end xor_gate;  
architecture xorLogic of xor_gate is  
begin  
    Y <= A xor B;  
end xorLogic;
```

2017

1 (a)

- ▼ Represent $(-200)_{10}$ and $(200)_{10}$ using sign-magnitude, 1's complement and 2's complement binary form #2020 (same but different number)

1 (b)

- ▼ Represent $(275)_{10}$ and $(641)_{10}$ in BCD and then perform BCD addition. Check your result by converting back to decimal
#revised-1

$$(275)_{10} = ()_{BCD}$$

$$= (0010 \ 0111 \ 0101)_{BCD}$$

$$\begin{array}{r} 8421 \\ 0010 = 2 \\ 0111 = 7 \\ 0101 = 5 \end{array}$$

$$(641)_{10} = ()_{BCD}$$

$$\begin{array}{r} 8421 \\ 0110 = 6 \\ 0100 = 4 \\ 0001 = 1 \end{array}$$

$$(0110 \ 0100 \ 0001)_{BCD}$$

$$\begin{array}{r} \text{carry} \swarrow \\ \begin{array}{r} 0010 \quad 0111 \quad 0101 \\ + 0110 \quad 0100 \quad 0001 \\ \hline 1011 \quad 0110 \\ + 0110 \\ \hline 1001 \quad 0001 \quad 0110 \end{array} \end{array}$$

$$\begin{array}{r} 8421 \\ 1011 = 11 \end{array}$$

$$\begin{array}{r} 8421 \\ 1001 = 9 \\ 0001 = 1 \\ 0110 = 6 \end{array}$$

$$(1001 \ 0001 \ 0110)_{BCD} = 916$$

$$\begin{array}{r} 641 \\ 275 \\ \hline 916 \end{array}$$

$\therefore$ It is justified.

1(c)

▼ Construct a BCD adder to add two 4-bit BCD code groups using 4-bit parallel adders and discuss its operation with suitable examples

2(a)

▼ Design a combinational logic circuit to compare two 4-bit binary numbers A and B and to generate the outputs $A < B$, $A = B$ and $A > B$

2 (b)

- ▼ Design a combinational circuit to convert BCD code to 7-segment code

2 (c)

- ▼ Design a combinational logic using suitable multiplexer to realize the following boolean expression: $Y = AD' + B'C + BC'D'$

3 (a)

- ▼ What is priority encoder? Design a 4-line to 2-line priority encoder with active HIGH inputs and outputs with priority assigned to the higher order data input lines

A priority encoder is a digital circuit that encodes multiple inputs into a binary code based on the highest-priority input. In other words, it determines which of the inputs has the highest priority and outputs a code indicating its position.

https://www.youtube.com/watch?v=gnoqn705LBg&ab_channel=ALLABOUTELECTRONICS

3 (b)

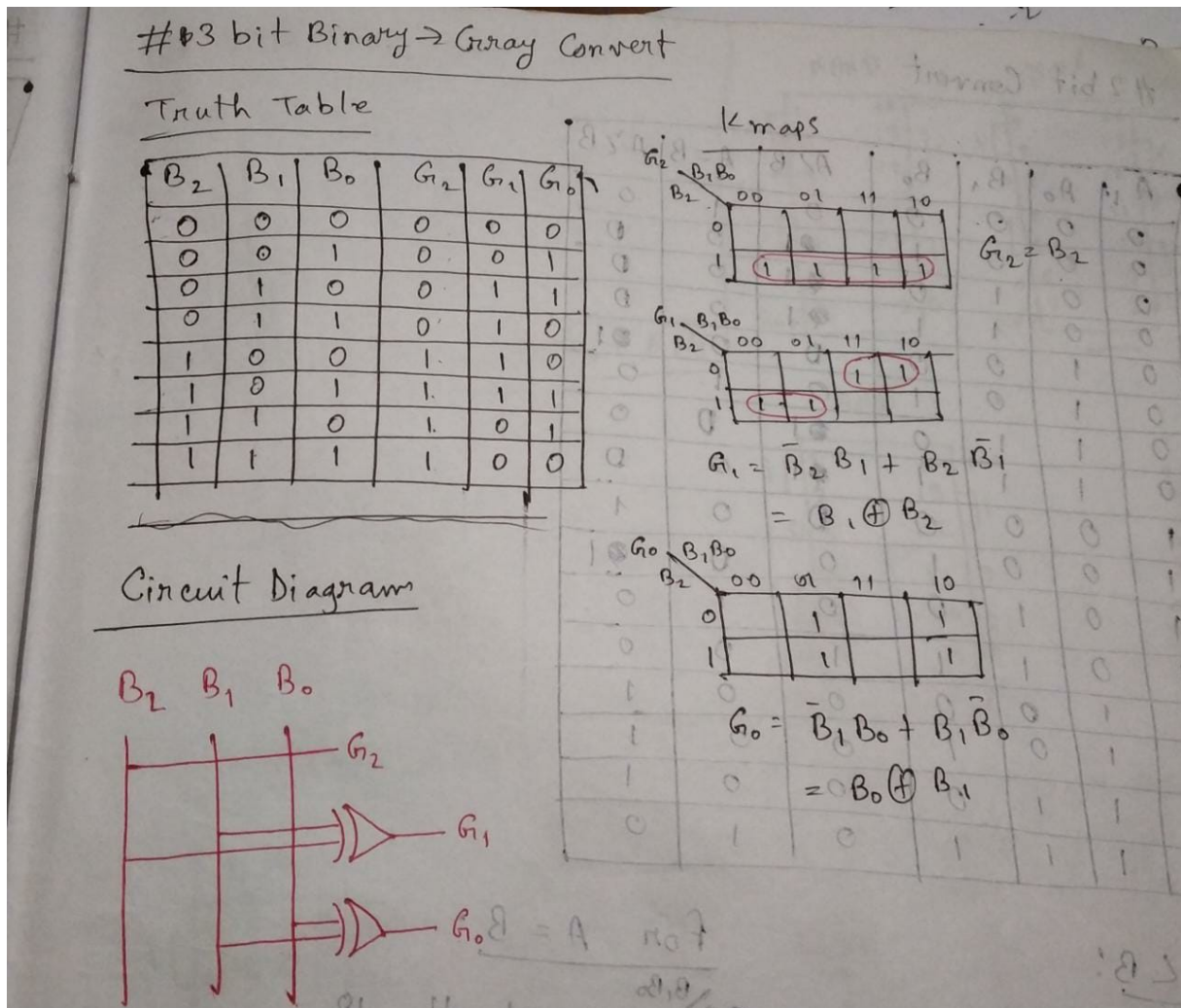
- ▼ Implement a full-adder circuit using a 3-to-8 decoder #2020

4 (a)

- ▼ What is code converter? #2014 **#revised-1** Draw the combinational logic circuit for binary to gray code conversion

Code Converter:

A code converter is a combinational digital circuit that transforms data from one binary code to another.

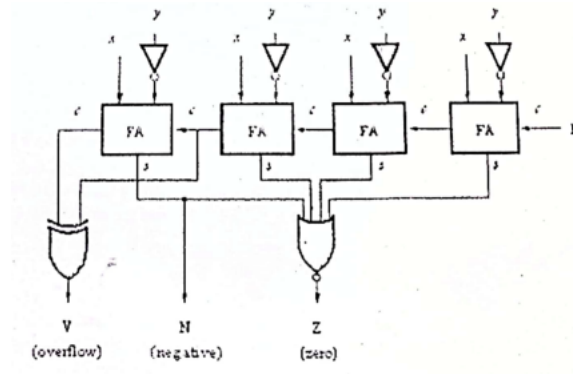


4 (b)

▼ Design and explain a 2-bit binary multiplier combinational logic circuit

4 (c)

In computer computations it is often necessary to compare numbers. Two four-bit signed numbers, $X = x_3x_2x_1x_0$ and $Y = y_3y_2y_1y_0$, can be compared by using the subtractor circuit in Figure, which performs the operation $X - Y$. The three outputs denote the following: $Z=1$ if the result is 0, otherwise $Z=0$; $N=1$ if the result is negative otherwise $N=0$; $V=1$ if arithmetic overflow occurs, otherwise $V=0$. Show how Z, N , and V can be used to determine the cases $X = Y, X < Y, X \leq Y, X > Y$ and $X \geq Y$



▼ Answer

5 (a)

▼ What is shift register? Design a 4-bit right shift register with parallel load of data

5 (b)

▼ What is counter? Differentiate between asynchronous and synchronous counter. List the applications of counter

5 (c)

▼ Draw the truth table and design a logic diagram of a MOD-10 ripple counter. Explain its operation with timing diagram

6 (a)

▼ Design a synchronous counter with JK flip-flop that will count the sequence as 0,2,4,6, and repeat

6 (b)

▼ Design a MOD-8 synchronous up/down counter and explain its operation with timing diagram

6 (c)

▼ What is presettable counters?

7 (a)

▼ Design a magnetic core RAM using 2×4 decoder (consider 2-bits input and 2-bits output). Discuss the read/write operation

7 (b)

▼ Distinguish between PLA and PAL in terms of design architecture

7 (c)

▼ Define the different hardware of FPGA. Draw the basic FPGA architecture. Give an example

8 (a)

▼ What is hardware description language? What are the requirements of a good HDL?

8 (b)

- ▼ Briefly describe the salient features of VHDL and Verilog

8 (c)

- ▼ What are the advantages of using JAVA HDL over Verilog?

2016

1 (a)

- ▼ Design a 4-bit binary adder circuit with necessary figures

1 (b)

- ▼ What do you mean by the terms 'carry generate' and 'carry propagate'? Define those terms with a full adder circuit and necessary equations

In digital circuit design, "carry generate" and "carry propagate" are terms used to describe the flow of carry signals in binary addition circuits.

A full adder circuit is a combinational logic circuit that adds three input bits (a, b, and c) and produces two output bits, sum (s), and carry (c_out). The carry bit allows multiple full adders to be combined to add larger binary numbers.

The equations for a full adder are as follows:

$$s = a \text{ XOR } b \text{ XOR } c_{\text{in}}$$

$$c_{\text{out}} = (a \text{ AND } b) \text{ OR } (c_{\text{in}} \text{ AND } (a \text{ XOR } b))$$

Now, let's define the terms "carry generate" and "carry propagate."

- Carry Generate: When two bits are added together, a carry-out (c_out) will be generated if both bits are 1. In other words, when both inputs are 1, the output will be 1, indicating that a carry is generated. This is known as the Carry Generate (G) function, which can be expressed as:

$$G = a \text{ AND } b$$

- Carry Propagate: A carry can also be propagated from a lower-order bit to a higher-order bit. In other words, if there is a carry-in (c_in) from a previous stage, it will propagate to the next stage, provided that at least one of the inputs is 1. This is known as the Carry Propagate (P) function, which can be expressed as:

$$P = a \text{ OR } b$$

Therefore, the carry-out (c_out) of a full adder can be represented as a combination of the Carry Generate (G) and Carry Propagate (P) functions, as follows:

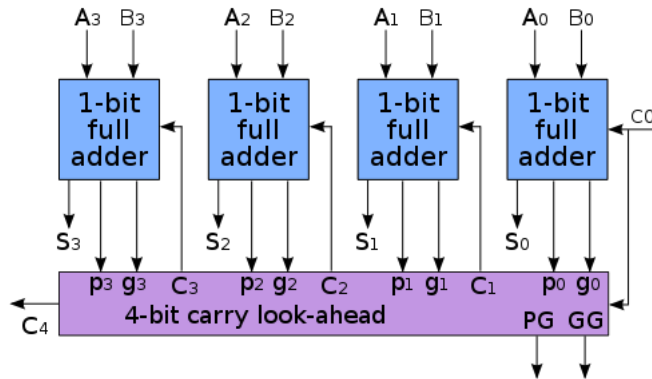
$$c_{\text{out}} = G + (P \text{ AND } c_{\text{in}})$$

This equation says that the carry-out is the result of either generating a carry or propagating a carry, depending on the input values and the carry-in value.

1 (c)

- ▼ Define lookahead carry-generator. Design a 4-bit carry-lookahead adder #revise-1

A lookahead carry generator is a digital circuit that produces carry signals for multiple stages of an adder circuit simultaneously. It can compute all the carry signals in parallel, making it faster and more efficient.



2 (a)

- ▼ Draw a 2-input AND gate and a 2-input OR gate using Transistor-Transistor logic

2 (b)

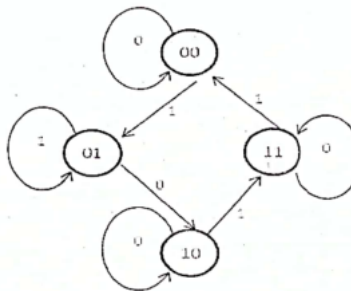
- ▼ There are Four switches to control a water pump. The water pump is turned on for the following conditions (i) if switch 1 is turned OFF but both switch 2 and 3 are turned ON, or, (ii) if switch 4 is turned ON but both switch 2 and 4 are turned OFF. Draw the truth table, derive the switching functions, minimize the switching functions by using K-map and draw that logic-gate diagram (circuit)

2 (c)

- ▼ What are the differences between '7400' and '74LS00'?

3 (a)

Design a clocked sequential circuit that counts from 00 to 11 with JK flip-flop whose state diagram is given below:



- ▼ Answer

3 (b)

- ▼ Define latch and flip-flop. Design a clocked master-slave JK flip-flop

3 (c)

- ▼ Define state reduction problem. What is the benefit by considering don't care terms in designing a sequential circuit?

4 (a)

Design the sequential circuit using register and ROM whose next state and outputs are defined by the following equations:

$$A_1(t+1) = \sum(4, 6)$$

$$A_2(t+1) = \sum(1, 3, 5, 7)$$

$$y(A_1, A_2, x) = \sum(3, 7)$$

▼ Answer

4 (b)

▼ Design a 2-input serial adder using a sequential logic procedure

4 (c)

▼ Discuss the difference between serial and parallel modes of operation in a register

Serial Mode	Parallel Mode
Bits are transferred sequentially, one after the other.	Multiple bits are transferred simultaneously in parallel.
Typically requires fewer clock cycles for data transfer.	Requires more clock cycles for data transfer
Can have higher data transfer rates.	Data transfer rates are lower compared to serial mode.
Requires fewer wires or lines for data transfer.	Requires more wires or lines for parallel data transfer.
cost-effective due to reduced hardware.	Higher cost due to additional components and connections.
Suitable for serial communication	Ideal for parallel data processing

5 (a)

▼ Draw the logic diagram of a 4-bit Ripple counter and explain its operation

5 (b)

▼ What do you mean by negative-edge triggered FF, positive edge triggered FF and level triggered FF. Explain with diagrams

5 (c)

▼ Draw the logic diagram of a 4-bit Johnson counter and explain its operation

6 (a)

▼ **Design an integrated circuit memory using 2×4 decoder (consider 2-bit input and 2-bit output)**

6 (b)

A combinational circuits is defined by the following functions:

$$F_1(A, B, C) = \sum(3, 4, 6, 7)$$

$$F_2(A, B, C) = \sum(0, 2, 5, 8)$$

Implement the circuit with a PLA having 3 inputs, 4 product terms and 2 outputs

▼ Answer

7 (a)

▼ Describe the abstraction levels of Verilog HDL

7 (b)

▼ Write the input-output connection rules of Verilog HDL with example

7 (c)

What is the difference between the following two lines of Verilog code?

i. #5 a=5;

ii. a=#5 b;

▼ Answer

8 (a)

▼ Write Verilog HDL programs including the design and stimulus blocks for (i) 4-1 multiplexer, (ii) 4-bit full adder

2015

1 (a)

▼ Why combinational logic circuit is needed? **Design a BCD to Excess-4 code converter and draw its logic diagram**

Combinational logic circuits are essential in digital systems for processing input data and producing output based solely on the current input values. They are called "combinational" because the output depends only on the combination of input values.

1 (b)

▼ Why carry-lookahead adder is faster than carry propagation adder. Draw the logic diagram of a 4-bit lookahead carry generator. Construct a 4-bit carry-lookahead adder with a lookahead carry generator

Carry-lookahead adders are faster than carry propagation adders because they use a parallel structure that can generate the carry signals for all of the bits in a binary number simultaneously. This allows them to avoid the ripple carry delay that is characteristic of carry propagation adders, which must wait for the carry signal from the previous bit before computing the next bit's sum and carry.

In other words, carry-lookahead adders use a more efficient mechanism for generating carry signals, which reduces the amount of time required for addition operations. Carry propagation adders, on the other hand, rely on a sequential process of propagating carries from one bit to the next, which can result in longer computation times for larger binary numbers.

<https://images.app.goo.gl/vg4Nr4EDxM2fU7Ei9>

2 (a)

▼ What is the difference between a decoder and a de-multiplexer? Implement a full adder circuit with a decoder and two OR gate (using 3×8 decoder) #2020

Feature	Decoder	Demultiplexer
Number of inputs	Usually has fewer inputs than outputs	Usually has equal number of inputs and outputs
Outputs	Generates all possible combinations of outputs	Routes input to one of many possible outputs
Functionality	Converts encoded inputs into binary outputs	Distributes a single input among multiple outputs
Implementation	Implemented using logic gates	Can be implemented using multiplexers
Example	Binary to decimal decoder	1-to-4 line demultiplexer

2 (b)

▼ Define multiplexer. Implement $F(A, B, C, D) = \sum(1, 2, 4, 8, 9, 15)$ with a 8×1 MUX

Consider a synchronous sequential binary logic circuit with one input A and one output Z. Suppose A denotes single bit binary number and the input sequence is divider per every three clock cycle. Here, each division is called an interval. We should like to design a sequential circuit on or more '1's. Table-1 illustrates an example of the action

Clock	C	1	2	3	4	5	6
Input	A	0	0	0	1	0	1
Output	Z	0	0	0	0	0	1

Table-1

3 (a)

- ▼ Draw the state transition diagram for the circuit

3 (b)

- ▼ Draw the state transition table for the circuit

3 (c)

- ▼ Draw the Karnaugh map for the circuit

3 (d)

- ▼ Design the synchronous sequential circuit and draw it. You can on;y use the symbols for AND,OR, NOT, D flip-flop, and JK flip-flop

4 (a)

- ▼ Distinguish between flip-flop and latch. Explain clocked SR flip-flop with logic diagram, function table and characteristics equation

4 (b)

Complete the diagram of the sequential circuit whose sequence is given below:

State	a	a	b	c	d	e	f	f	g	f	g
Input	0	1	0	1	0	1	1	0	1	0	0
Output	0	0	0	0	0	1	1	0	1	0	0

- ▼ Draw the logic diagram using SR flipflops

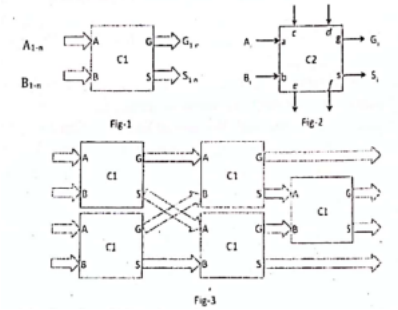
5 (a)

- ▼ What is shift register? Desgin a 4-bit bidirectional shift register with parallel load

5 (b)

- ▼ What is ripple counter? Draw the logic diagram of a BCD ripple counter with timing diagram

A circuit C1 shown in Fig-1 accepts n-bit unsigned binary numbers A and B as input and produce G and S as output, where G is the greater and S is the smaller between A and B respectively. When $A=B$, we have $G = S = A = B$. Fig-2 shows a circuit C2 processes numbers of one binary digit. Suppose that cascading n units of circuit C2 implement a circuit C1. The inputs c and d of circuit C2 represent that $A < B$ and $A > B$ respectively for more significant digits than processed by the circuit. The outputs e and f represent that $A < B$ and $A > B$ hold respectively including the digits being processed. G_i and S_i respectively represent i^{th} bit of greater and smaller number



6 (a)

- ▼ Construct a truth table for circuit C2. Note that '1' represents *true* and '0' represents false

6 (b)

- ▼ Draw a circuit diagram of C2 by using only AND, OR and NOT gate

6 (c)

- ▼ Draw a block diagram of circuit C1 for comparing two 4-bit binary numbers by using four C2 circuit blocks

6 (d)

- ▼ As shown in Fig-3, four input numbers are sorted by connecting several units of circuit C1. Following Fig-3, show an example circuit for sorting three inputs

7 (a)

- ▼ Distinguish between SRAM and DRAM. Briefly explain the internal organization of a 64×4 RAM

7 (b)

- ▼ What are the major differences between CPLDs and FPGAs? Discuss FPGA architecture in brief

8 (a)

- ▼ State port connection rules in Verilog HDL

8 (b)

- ▼ Explain min, max and typ delays in the gate-level design with respect to Verilog HDL

8 (c)

- ▼ Describe a 8-to-1 line Multiplexer module in Verilog HDL

2014

1 (a)

Define code converter? Design 3 bit Gray Code to binary converter and explain the operation #2017

1 (b)

Design an 8-bit magnitude comparator using 74HC85 4-bit magnitude comparator and hence describe the operation of the following eight-bit comparison

A7....A0=10101111 and B7.....B0=10101001

2 (a)

Design a 40bit binary parallel adder/subtractor circuit and explain the operation with an example

2 (b)

▼ There are two types of adder, one is ripple carry and other is carry look-ahead. What will be the good design of 32-bit adder? Explain your answer

For a 32-bit adder, the most efficient design would be a carry look-ahead adder. This is because ripple carry adders have a critical path delay that grows linearly with the number of bits being added, which can result in long propagation delays and limit the maximum clock frequency.

In contrast, carry look-ahead adders use a parallel structure to generate carry signals for all of the bits in a binary number simultaneously, allowing for faster addition operations. The carry lookahead adder consists of two main components: the "generate" and "propagate" functions. The generate function determines whether a carry will be generated when adding two bits, while the propagate function determines whether a carry from a previous bit should be propagated to the current bit.

By using these functions to compute the carry signals in parallel across all the bits in the number, the carry lookahead adder can avoid the ripple carry delay that is characteristic of ripple carry adders, allowing for faster addition operations.

Therefore, for a 32-bit adder, a carry-lookahead adder would be the best choice for achieving high-speed addition operations.

3 (a)

Design a binary multiplier of two numbers A and B (two bit each), $C=A \times B$. Explain its operation

3 (b)

▼ Explain an application of magnitude comparator with a digital circuit

A magnitude comparator is a digital circuit that compares the relative magnitudes of two binary numbers and produces an output signal indicating whether one number is greater than, less than or equal to the other number. One application of magnitude comparators is in binary arithmetic operations such as addition and subtraction.

For example, in a microprocessor's arithmetic logic unit (ALU), magnitude comparators are used to compare the operands being added or subtracted to determine the appropriate operation and to detect overflow conditions. The ALU receives two operands and a control signal indicating which operation to perform (addition or subtraction), and uses a magnitude comparator to compare the operands and generate the necessary carry and borrow signals.

4 (a)

Design a 1-line-to-8 line demultiplexer and discuss the operation of the circuit

4 (b)

From the following

Synchronous Systems	Asynchronous Systems
Uses a global clock signal for synchronization.	Does not rely on a global clock signal for synchronization.
Requires clock distribution	No need for clock distribution
More complex due to clock distribution	Relatively simpler due to lack of clock dependencies.
Higher power consumption	Potentially lower power consumption
Performance is predictable and can be optimized.	Performance may vary depending on input conditions.

CPUs, microcontrollers, synchronous digital systems.	Asynchronous communication, event-driven systems.
--	---